Multi-Cloud Scalable Rendering Infrastructure Design

Design a **scalable, multi-cloud infrastructure** capable of dynamically provisioning compute environments for a rendering platform.

The system must support **on-demand provisioning of virtual machines with specific software and plugin configurations**, based on user input.

The platform must allow users to:

- Select **Software** (Maya, 3ds Max, etc.) + **version**
- Select **Render engine** (Arnold, Redshift, V-Ray, etc.) + **version**
- Request **machine type** (e.g., 48 vCPU) and **number of machines** (1 to 10,000)

Architecture Design

The system is split into a centralized Control Plane and distributed Data Planes to ensure scalability and reliability .

High-Level Architecture:

Control Plane (central, hosted in AWS)

- Handles user requests, authentication, orchestration, compatibility validation, and state management.
- Contains API Gateway + Auth, Queue, Orchestrator, DynamoDB, SDK Abstraction layer, and monitoring (Prometheus + Grafana).

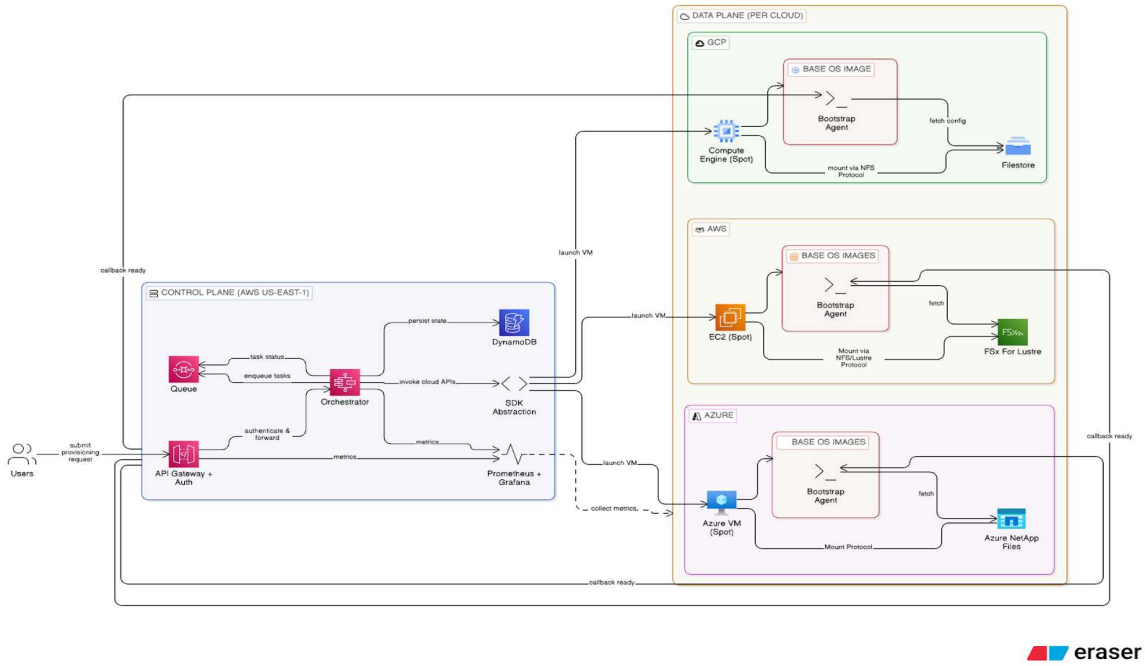
Data Plane (runs independently in each cloud)

- AWS: EC2 Spot instances + Base OS Image + Bootstrap Agent + FSx for Lustre.
- GCP: Compute Engine Spot instances + Base OS Image + Bootstrap Agent + Filestore.
- Azure: Azure VM Spot instances + Base OS Image + Bootstrap Agent + Azure NetApp Files.

Multi-cloud Interaction

The Control Plane uses a thin SDK Abstraction layer to call native cloud APIs (no vendor lock-in). All heavy file operations (installers and config) stay inside the target cloud via mounted file storage, eliminating egress/ingress charges.

Diagram



eraser

link;https://app.eraser.io/workspace/HpZfipqTGbEdHwXi841o?origin=share

Provisioning Strategy ("< 5 Minute"):

To eliminate egress charges and keep provisioning under 5 minutes, each cloud region contains a local **High-Performance Shared File System (e.g., AWS FSx)**. This storage acts as a 'Read-Only' library for all 150+ software combinations. At boot, the Bootstrap Agent mounts the required paths based on metadata, allowing the render engine to execute immediately without a local installation phase.
Thin Base Images: Instead of separate images for every combo, we use one "Thin" Base Image per cloud provider containing only the OS and pre-installed GPU drivers .
(Pre-installing drivers is essential as they take >10 minutes to install at boot).
Zero-Download Software Access
On boot, the VM does **not** download any software. Instead, the Bootstrap Agent performs a **high-speed network mount** of the regional file system:

AWS → FSx for Lustre

GCP → Filestore (with NFS/Lustre protocol)

Azure → Azure NetApp File.

Parallel Bootstrapping
By mounting the software via the network, the machine is ready for rendering as soon as the OS finishes booting (2-3 minutes).
The entire flow remains the same as shown in the architecture diagram: Control Plane launches Spot VMs → Bootstrap Agent mounts the file store → environment is configured → READY callback.

Configuration Management: (How we handle the 150+ software combos).

Software & Plugin Management

Centralized Library: All 150+ software and plugin versions are stored as binaries on the **regional shared file system** (FSx for Lustre / Filestore / NetApp Files).

Compatibility Handling: The Orchestrator uses a **DynamoDB-backed compatibility matrix** to verify that the selected plugin version is compatible with the selected software version **before** the VM is even launched. Invalid combinations are rejected instantly.

Passing Configuration to Machines

Metadata Injection: The Orchestrator passes the selected versions (e.g., maya_v=2015, arnold_v=4.2.1, plugins list) to each VM using **Instance Metadata / User Data / Tags**.

Environment Orchestration: The Bootstrap Agent reads these metadata values and dynamically sets system environment variables (PATH, LD_LIBRARY_PATH, MAYA_MODULE_PATH, etc.) so the rendering engine immediately locates the mounted tools on the network file system.

Cost & Multi-Cloud Optimization:

Assumption: Regional storage (FSx/Filestore/NetApp Files) is replicated once per cloud provider. This completely eliminates egress/ingress charges during both bootstrapping and rendering.

Storage vs. Image Cost: Storing one shared centralized library is **significantly cheaper** than maintaining 150+ large VM disk images (golden images).

Compute Optimization: The system defaults to **Spot Instances** (AWS/GCP) or **Spot VMs** (Azure) to reduce compute costs by up to 90%, with automatic fallback to on-demand if capacity is unavailable.

Trade-offs

Slightly higher network dependency on the file store (mitigated by using ultra-low-latency Lustre-style systems).

Initial setup of the shared library is a one-time cost (then near-zero incremental cost for new versions).

Summary

This design provides a high-performance, cost-optimized solution for dynamically provisioning 150+ combinations of 3D rendering software and plugins across AWS, GCP, and Azure . By decoupling the software layer from the machine image, the system achieves a provisioning time of less than **5 minutes** while eliminating the need for expensive and unmanageable "Golden Images" .This JIT Configuration design strictly satisfies all constraints: multi-cloud support, no per-combination images, ≤5-minute provisioning, minimized storage, zero egress charges, and efficient Spot compute usage